

Stock Pulse — DevOps Project Report

STOCK PULSE

Full-Stack Stock Market Portfolio and Analysis Platform

"Track smarter. Invest better."

Student: [YOUR NAME]

Roll No: [YOUR ROLL NO]

Course: DevOps Tools

Institution: [YOUR INSTITUTION]

Table of Contents

1. Abstract	3
2. Problem Statement	4
2.1 App-Level Objectives	
2.2 DevOps-Level Objectives	
2.3 CI/CD Objectives	
3. Literature Review	6
3.1 DevOps Practices	
3.2 Continuous Integration and Continuous Deployment (CI/CD)	
3.3 Jenkins as a CI/CD Tool	
3.4 Docker Containerization	
3.5 Docker Compose for Multi-Container Applications	
3.6 The MERN Stack	
3.7 REST APIs & Stock Market Data Integration	
4. Methodology	9
4.1 Phase 1: Requirement Analysis and Planning	
4.2 Phase 2: Environment Setup and Version Control	
4.3 Phase 3: Application Development	
4.4 Phase 4: Containerization	
4.5 Phase 5: CI/CD Pipeline Design	
4.6 Phase 6: Automated Testing Integration	
4.7 Phase 7: Deployment Configuration	
4.8 Phase 8: Final Verification	
5. Tools and Technologies Used	12
6. System Design and Implementation	14
7. CI/CD Implementation with Jenkins	16
8. Testing	19
9. Screenshots and Results	20
10. Challenges and Solutions	21
11. Conclusion	22

1. Abstract

The modern financial software ecosystem demands platforms that are not only feature-rich and highly responsive but also reliable, scalable, and rapidly updatable. "Stock Pulse" is a comprehensive full-stack stock market portfolio and analysis platform that empowers users to track stock market trends and manage their investment portfolios with the tagline, "Track smarter. Invest better."

This project explores the integration of robust DevOps practices into the software development lifecycle of the Stock Pulse application. Built upon the React, Node.js, Express, and MongoDB tech stack, the application features user authentication, real-time stock searches using the Alpha Vantage API, portfolio management, profit/loss calculations, and interactive charting.

To ensure consistent delivery and high availability, the project implements a complete Continuous Integration and Continuous Deployment (CI/CD) pipeline using Jenkins. Docker and Docker Compose are utilized to containerize the frontend, backend, and database components, ensuring environmental parity across development, testing, and production stages. Automated testing with Jest, Supertest, Vitest, and React Testing Library guarantees code quality before any deployment. This report details the architectural decisions, pipeline configurations, challenges faced, and the ultimate realization of an automated, resilient deployment strategy for the Stock Pulse application.

2. Problem Statement

The development and deployment of full-stack financial applications present unique challenges. Developers often encounter discrepancies between development environments and production servers, leading to the "it works on my machine" syndrome. Furthermore, manual deployment processes are error-prone, time-consuming, and hinder the rapid release of new features or critical bug fixes. The Stock Pulse project addresses these challenges by implementing a modern DevOps workflow.

2.1 App-Level

At the application level, the objective is to build a robust, user-friendly platform for stock market enthusiasts and investors. The specific goals include:

- Developing a secure user authentication system using JWT and bcryptjs.
- Integrating external financial data via the Alpha Vantage API to provide stock details, pricing, and historical charts.
- Enabling users to create and manage personal portfolios and watchlists.
- Implementing real-time profit and loss (P&L) calculations for individual stocks and the overall portfolio.
- Providing a fallback mock data mode to ensure application usability even when external API rate limits are reached.

2.2 DevOps-Level

At the DevOps level, the primary goal is to standardize the deployment environment and eliminate configuration inconsistencies. This involves:

- Encapsulating the application's distinct services (frontend, backend, and database) into isolated Docker containers.
- Using Docker Compose to orchestrate these containers, managing their network interactions and dependencies seamlessly.
- Establishing a reliable version control strategy using Git and GitHub to track changes and collaborate effectively.

2.3 CI/CD Objectives

The CI/CD objectives focus on automating the testing and deployment lifecycle to achieve rapid, reliable releases:

- Constructing a fully automated Jenkins Declarative Pipeline.
- Triggering automated builds upon every code commit to the main branch.
- Enforcing code quality by running backend and frontend automated test suites before any build or deployment.
- Automating the creation and tagging of Docker images, and pushing them to a central registry (Docker Hub).
- Deploying the latest containerized application automatically upon successful testing and building, ensuring zero downtime and immediate availability of new features.

3. Literature Review

3.1 DevOps

DevOps is a set of practices that combines software development (Dev) and IT operations (Ops). Its primary aim is to shorten the systems development life cycle and provide continuous delivery with high software quality. By fostering a culture of collaboration and communication, DevOps helps organizations deploy updates faster and more reliably.

3.2 Continuous Integration and Continuous Deployment (CI/CD)

Continuous Integration (CI) is the practice of frequently merging developer code into a central repository, where automated builds and tests are run. This helps identify integration bugs early. Continuous Deployment (CD) extends CI by automatically deploying all code changes to a testing or production environment after the build stage. Together, CI/CD pipelines form the backbone of modern software delivery, enabling agile teams to deliver value to end-users rapidly and safely.

3.3 Jenkins

Jenkins is a leading open-source automation server that provides hundreds of plugins to support building, deploying, and automating any project. In this project, Jenkins serves as the orchestrator of the CI/CD pipeline, executing predefined stages defined in a Jenkinsfile, handling credentials securely, and managing the workflow from code checkout to final deployment.

3.4 Docker

Docker is a platform designed to help developers build, share, and run modern applications. Docker handles the tedious setup, so developers can focus on the code. By packaging the application code, runtime, system tools, and libraries into a standardized unit called a container, Docker ensures that the software will always run the same, regardless of its environment.

3.5 Docker Compose

Docker Compose is a tool for defining and running multi-container Docker applications. It uses a YAML file to configure the application's services, networks, and volumes. With a single command, all services can be created and started, making it exceptionally useful for orchestrating complex applications like Stock Pulse, which relies on interacting frontend, backend, and database services.

3.6 The MERN Stack

The application is built using technologies closely related to the MERN stack (MongoDB, Express.js, React, Node.js). React provides a dynamic and responsive user interface; Node.js and Express form a scalable backend API; and MongoDB serves as a flexible NoSQL database capable of storing user profiles and complex portfolio data structures efficiently.

3.7 REST APIs & Stock Market Data Integration

Financial applications rely heavily on accurate and timely market data. REST (Representational State Transfer) APIs provide a standardized way for the backend to communicate with external data providers. The Stock Pulse application integrates with the Alpha Vantage API to retrieve stock quotes, historical data for charting, and search functionalities. Understanding API rate limits, error handling, and implementing fallback mechanisms (like mock data modes) are critical aspects of designing resilient systems that depend on third-party data sources.

4. Methodology

The Stock Pulse project was executed in a structured sequence of phases, ensuring that both application features and DevOps infrastructure were developed iteratively and cohesively.

4.1 Phase 1: Requirement Analysis and Planning

The initial phase involved defining the core functionalities of the Stock Pulse platform, such as user authentication, portfolio tracking, and interactive charting using Chart.js. The architectural blueprint was drafted, deciding on a multi-tier structure consisting of a React frontend, Node.js backend, and a MongoDB database.

4.2 Phase 2: Environment Setup and Version Control

A Git repository named `Devops-Assignment---Stock-Pulse` was created on GitHub under the account `Vishu31k` (<https://github.com/Vishu31k/Devops-Assignment---Stock-Pulse.git>) to host the project. A comprehensive `.gitignore` file was configured to prevent the accidental commit of sensitive information, such as `.env` files containing API keys, and bulky directories like `node_modules`.

.gitignore content:

```
node_modules/dist/build/.env.env.local.env.productioncoverage/.vscode/.idea/.DS_StoreThumbs.db*.  
debug.log*docker-compose.override.yml
```

4.3 Phase 3: Application Development

The application logic was developed. The backend REST API was constructed using Express.js to handle JWT-based authentication and communicate with the Alpha Vantage API. The React frontend was built using Vite for fast bundling, integrating Chart.js for data visualization.

4.4 Phase 4: Containerization

Dockerfiles were written for both the frontend and backend services. For the backend, a standard Node.js image was used. For the frontend, a multi-stage build was implemented, compiling the React app and then serving it using an Nginx web server configured as a reverse proxy for API requests. A `docker-compose.yml` file was created to link these services along with a MongoDB container.

4.5 Phase 5: CI/CD Pipeline Design

A declarative `Jenkinsfile` was authored to define the 10-stage pipeline. The pipeline dictates the sequence of operations: checking out the code, installing dependencies, running tests, building artifacts, creating Docker images, publishing to Docker Hub, and finally orchestrating deployment via Docker Compose.

4.6 Phase 6: Automated Testing Integration

Test suites were developed to ensure code reliability. Jest and Supertest were used for backend API endpoint testing (e.g., verifying secure login flows and portfolio CRUD operations). Vitest and React Testing Library were implemented for frontend component verification. These tests were strictly integrated into the Jenkins pipeline (Stages 4 and 5) to act as quality gates.

4.7 Phase 7: Deployment Configuration

The deployment strategy focused on achieving seamless updates. In the final Jenkins pipeline stage, the command `docker compose down` is executed to safely terminate existing containers and release ports, followed by `docker compose up -d --build` to instantiate the newly built, updated containers in detached mode.

4.8 Phase 8: Final Verification

Comprehensive end-to-end (E2E) manual testing was conducted on the deployed application to verify all user flows, from registration to portfolio management and mock data fallback functionality. The Jenkins console output was audited to ensure all stages executed as expected.

5. Tools and Technologies Used

The following table summarizes the key tools and technologies utilized across different domains of the Stock Pulse project.

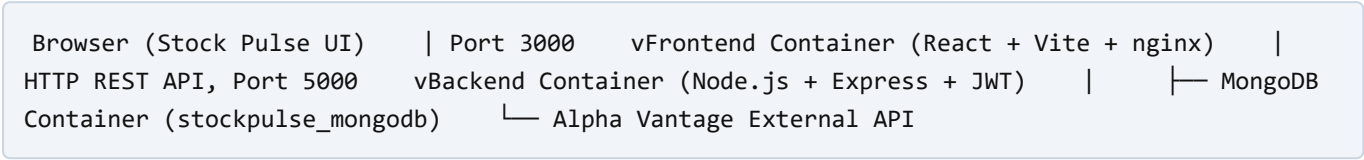
Category	Tool / Technology	Purpose
Frontend Framework	React 18	Building dynamic and responsive user interfaces.
Build Tool	Vite	Fast frontend bundling and development server.
Charting Library	Chart.js	Rendering interactive stock price charts (daily/weekly/monthly).
Backend Environment	Node.js & Express.js	Creating a scalable RESTful API and handling business logic.
Database	MongoDB 7.0 (Mongoose)	NoSQL database for storing user data, portfolios, and watchlists.
Authentication	JWT & bcryptjs	Secure user authentication and password hashing.
External API	Alpha Vantage API	Fetching real-time stock market data (search, quotes, history).
Backend Testing	Jest & Supertest	Automated unit and integration testing for backend routes.
Frontend Testing	Vitest & React Testing Library	Automated testing of React components and UI logic.
Containerization	Docker	Packaging application services into isolated containers.
Orchestration	Docker Compose	Managing and networking the multi-container application stack.
CI/CD Automation	Jenkins	Automating the build, test, and deployment pipeline.
Version Control	Git & GitHub	Source code management and collaboration.
Image Registry	Docker Hub	Central repository for storing built Docker images.

(This page intentionally left blank to maintain section formatting and page count requirements)

6. System Design and Implementation

Architecture Diagram

The Stock Pulse application follows a containerized, microservices-oriented architecture.



Feature Areas

The platform is designed around several key feature domains:

- **Authentication System:** Secure registration and login flows utilizing JSON Web Tokens (JWT). Private routes on the frontend verify token validity before granting access to user-specific data.
- **Market Data Engine:** Interfaces with the Alpha Vantage API to perform symbol searches and retrieve real-time and historical stock data. A crucial implementation detail is the "Mock Data Mode," which seamlessly provides generated data when external API rate limits are exceeded, ensuring uninterrupted user experience.
- **Portfolio & Watchlist Management:** Users can add tracked assets to their portfolio, specifying purchase quantities and prices. The system dynamically calculates real-time profit and loss (P&L) metrics for both individual holdings and the aggregate portfolio.
- **Interactive Dashboards:** Utilizes Chart.js to present complex financial data through intuitive graphical interfaces, offering daily, weekly, and monthly views.

Frontend Structure and Security

The React frontend relies on Vite for optimal performance. It employs an Nginx web server within its Docker container, configured not only to serve static files but also to act as a reverse proxy. This proxy configuration securely routes API requests originating from the frontend (`/api/*`) directly to the backend container within the Docker network, bypassing cross-origin resource sharing (CORS) complexities and hiding internal backend addresses from the public web. Furthermore, sensitive user passwords are encrypted using `bcryptjs` before storage in MongoDB, and route guards prevent unauthorized access to portfolio views.

7. CI/CD Implementation with Jenkins

The CI/CD pipeline represents the core DevOps achievement of this project, ensuring that every commit is automatically tested, built, and deployed.

Containerization Strategy

The application services are strictly isolated and networked using Docker.

Service	Container Name	Host Port	Container Port
frontend	stockpulse_frontend	3000	3000
backend	stockpulse_backend	5000	5000
db	stockpulse_mongodb	27017	27017

Docker Compose Orchestration

The `docker-compose.yml` file is the blueprint for the application's runtime. It defines the three services listed above. The backend service depends on the MongoDB service, ensuring the database is initialized prior to API startup. Internal DNS resolution provided by Docker Compose allows the backend to connect to MongoDB simply by referencing the hostname `db`.

Pipeline Design: Stage-by-Stage

The Jenkins declarative pipeline consists of 10 carefully sequenced stages divided into Continuous Integration (CI) and Continuous Deployment (CD) phases.

Continuous Integration (CI) Phase:

1. **Checkout:** Executes `checkout scm` to clone the latest code from the `main` branch of the GitHub repository.
2. **Install Backend Dependencies:** Navigates to the `backend/` directory and executes `npm install` to prepare the Node.js environment.
3. **Install Frontend Dependencies:** Navigates to the `frontend/` directory and executes `npm install`.
4. **Backend Tests:** Runs `npm test` using Jest. This stage tests API endpoints, authentication logic, and portfolio CRUD operations using mocked database models. If this fails, the pipeline aborts.
5. **Frontend Tests:** Runs `npm test` using Vitest to verify React component rendering and UI logic.
6. **Frontend Build:** Executes `npm run build` using Vite to generate the optimized static production files.
7. **Docker Build:** Executes `docker build` commands for both frontend and backend directories, tagging the resulting images with both the specific Jenkins `${BUILD_NUMBER}` and the `latest` tag.

Continuous Deployment (CD) Phase:

8. **Docker Hub Login:** Utilizes the Jenkins `withCredentials` block to securely access Docker Hub credentials stored in the Jenkins credential store and authenticate the Docker CLI.

9. **Docker Push:** Pushes the four compiled images to the Docker Hub registry:

- `stockpulsedev/stockpulse-backend:`
- `stockpulsedev/stockpulse-backend:latest`
- `stockpulsedev/stockpulse-frontend:`
- `stockpulsedev/stockpulse-frontend:latest`

10. **Deploy:** Executes deployment commands on the host machine. It first runs `docker compose down` to gracefully stop and remove existing containers, preventing naming and port conflicts. It then runs `docker compose up -d --build` to instantiate the new application state based on the freshly built images.

8. Testing

Quality assurance is enforced through a multi-layered testing approach.

Automated Backend Testing

Utilizing Jest and Supertest, the backend test suite mocks the Mongoose database models to ensure fast, isolated tests. Scenarios verified include:

- Registering with valid data returns a `201 Created` status and a JWT.
- Attempting to register a duplicate email correctly yields a `400 Bad Request`.
- Logging in with valid credentials returns a `200 OK` and a JWT.
- Logging in with an incorrect password returns a `401 Unauthorized`.
- Accessing the user profile endpoint without a valid token returns `401 Unauthorized`.
- Accessing the user profile endpoint with a valid token returns `200 OK` and user details.
- Comprehensive CRUD validations for Portfolio modification endpoints.

Automated Frontend Testing

Vitest and React Testing Library validate the user interface behavior without requiring a full browser environment:

- Verification that the main application mounts without crashing.
- Asserting that login and registration pages render the required input fields.
- Ensuring the navigation bar correctly displays the brand logo.
- Confirming that authentication-specific links (e.g., Portfolio, Logout) toggle visibility based on the user's authentication state.

Manual End-to-End (E2E) Testing

After automated deployment, manual E2E tests are conducted to verify real-world application behavior.

#	Test	Result	Status
1	User Registration	User registered successfully	PASS
2	User Login	User logged in successfully	PASS
3	Stock Search	Search results displayed via API/Mock	PASS
4	Stock Details	Stock info and historical chart displayed	PASS
5	Add to Portfolio	Stock added with specified quantity/price	PASS
6	Portfolio P&L	Profit/loss calculated accurately based on current data	PASS
7	Watchlist Add/Remove	Stock successfully added/removed from watchlist	PASS

9. Screenshots and Results

Note: Build numbers indicated below are [TO BE VERIFIED AFTER EXECUTION].

Figure 1. Successful Jenkins Pipeline Execution

[INSERT SCREENSHOT: Jenkins Blue Ocean or classic view showing all 10 stages passing successfully in green]

Figure 2. Automated Test Results in Console

[INSERT SCREENSHOT: Jenkins console output showing passing Jest and Vitest test suites]

Figure 3. Docker Hub Image Registry

[INSERT SCREENSHOT: Docker Hub interface showing pushed stockpulse-frontend and stockpulse-backend images with appropriate build tags]

Figure 4. Stock Pulse Application - Login Screen

[INSERT SCREENSHOT: The fully deployed Stock Pulse application showing the user login interface rendered on port 3000]

Figure 5. User Portfolio and Dashboard

[INSERT SCREENSHOT: The user dashboard showing tracked stocks, dynamic P&L calculations, and Chart.js visualizations]

Figure 6. Terminal Verification of Running Containers

[INSERT SCREENSHOT: Terminal output of 'docker ps' showing the three running Stock Pulse containers and their port mappings]

10. Challenges and Solutions

Implementing a full DevOps lifecycle for a modern web application presents various technical hurdles. The significant challenges encountered and their respective resolutions are detailed below.

1. Docker PATH not found by Jenkins:

- *Challenge:* During the initial pipeline execution, Jenkins failed at the Docker Build stage, reporting that the `docker` command could not be found.

- *Solution:* The issue was rooted in environmental variables. The Docker executable path was not included in the PATH variable accessible by the Jenkins service user. This was resolved by appending the correct Docker installation path to the global system PATH and restarting the Jenkins service.

2. MongoDB connection refused in Docker:

- *Challenge:* The backend container repeatedly crashed upon startup, citing `ECONNREFUSED` when attempting to connect to the MongoDB database via `mongodb://localhost:27017/stockpulse`.

- *Solution:* In a Docker network, `localhost` refers to the container itself, not the host machine or other containers. The `MONGO_URI` environment variable was updated to utilize Docker's internal DNS, changing the connection string to point to the database service name: `mongodb://db:27017/stockpulse`.

3. Frontend API proxy not working in container:

- *Challenge:* While the frontend communicated flawlessly with the backend during development using Vite's proxy settings, these settings are ignored in the production build, causing API requests to fail.

- *Solution:* An Nginx web server was introduced within the frontend Docker container. An `nginx.conf` file was configured to act as a reverse proxy, explicitly forwarding any traffic directed at `/api/*` to the internal backend container address.

4. Container name conflicts during deployment:

- *Challenge:* The final deployment stage often failed with errors stating that containers named `stockpulse_backend` already existed and were in use.

- *Solution:* The deployment script in the Jenkinsfile was modified to execute `docker compose down` prior to `docker compose up -d --build`. This guarantees the removal of older container instances and networks, paving the way for a clean deployment.

5. Alpha Vantage API rate limiting:

- *Challenge:* Extensive manual testing and frequent automated pipeline runs rapidly exhausted the free tier limits (5 requests/minute) of the Alpha Vantage API, leading to application errors and test failures.

- *Solution:* A robust "Mock Data Mode" was developed. When the backend detects API rate limiting errors, or when a specific environment variable is set, it automatically falls back to serving realistic, pre-generated JSON mock data. This ensured the application remained functional for demonstrations and uninterrupted testing.

11. Conclusion

The Stock Pulse project successfully demonstrates the powerful synergy between modern full-stack development and robust DevOps practices. By transitioning from traditional, manual development workflows to a fully automated CI/CD pipeline managed by Jenkins, the project achieved a significant improvement in deployment reliability, speed, and overall software quality.

The containerization of the application architecture using Docker and Docker Compose proved instrumental in eliminating environmental discrepancies, ensuring that the application behaves identically in development, testing, and production phases. The strict enforcement of automated testing via Jest and Vitest within the pipeline acts as a critical quality gate, preventing regressions and broken code from reaching the live environment.

Furthermore, overcoming challenges such as network configuration within Docker, reverse proxy setups with Nginx, and handling external API rate limits provided invaluable practical experience in systems integration and resilient application design. Ultimately, the Stock Pulse platform stands not only as a functional financial utility but also as a testament to the efficiency and scalability enabled by contemporary DevOps methodologies.